Федеральное государственное бюджетное образовательное учреждение высшего образования
«ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ ДВАЖДЫ ГЕРОЯ СОВЕТСКОГО СОЮЗА, ЛЕТЧИКА-КОСМОНАВТА А.А. ЛЕОНОВА»

Колледж космического машиностроения и технологий

«Допущен к защите»
заместитель директора колледжа по
учебной работе _____
Е.В. Антропова
«13» июня 2024 г.

Дипломный проект

РАЗРАБОТКА СИСТЕМЫ ОТСЛЕЖИВАНИЯ ТЕМАТИК КУРСОВЫХ И ДИПЛОМНЫХ ПРОЕКТОВ

Пояснительная записка

ДП.09.02.03.24.07ПЗ

Обучающийся группы П1-20 Головко М.Д.

Руководитель дипломного проекта Попов В.Н.

Результат защиты _____

Королев 2024 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....4

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ.....6

1.1 Описание предметной области.....6

1.2 Постановка задачи.....6

1.3 Анализ существующих программных решений.....12

2 ПРОЕКТНАЯ ЧАСТЬ.....13

2.1 Описание алгоритмов решаемых задач.....13

2.2 Обоснование выбора инструментов разработки.....14

2.3 Структура реализуемого web-приложения.....19

2.4 Описание сценариев взаимодействия с web-приложением.....26

2.5 Описание используемых процедур и функций.....28

3 ОРГАНИЗАЦИОННО-ЭКОНОМИЧЕСКАЯ ЧАСТЬ.....30

3.1 Расчёт стоимости разработки приложения.....30

3.2 Эксплуатационный раздел/Руководство пользователя.....36

3.3 Техника безопасности.....41

ЗАКЛЮЧЕНИЕ.....47

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ИНФОРМАЦИИ.....48

Приложение.....50

ВВЕДЕНИЕ

Данный проект — система отслеживания тематик курсовых и дипломных проектов — заключается в разработке веб-приложения с базой данных, которая будет хранить в себе тематики студенческих работ и все остальные необходимые элементы контекста: студентов, дисциплины, группы и квалификации. Ключевой функцией данной разработки является рекомендательная система сохранения уникальности тем.

Целью проекта является упрощение работы сотрудников учебного заведения, для которого разрабатывается проект.

Отдельными задачами проекта является разработка требований к проекту, разработка базы данных и приложения и разработка документации к системе.

В данной пояснительной записке рассмотрены следующие элементы ведения проекта:

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ — обобщает в себе всю работу перед непосредственно разработкой .

1.1. Описание предметной области — содержит в себе более подробное описание бизнес-среды, для которой разрабатывается проект

1.2. Постановка задачи — содержит в себе подробную постановку задачи и все этапы разработки.

1.3. Анализ существующих программных решений

2. ПРОЕКТНАЯ ЧАСТЬ — непосредственно разработка приложения

2.1. Описание алгоритмов решаемых задач — содержит в себе

2.2. Обоснование выбора инструментов разработки — содержит подробное описание среды и инструментов, использованных для создания проекта.

2.3. Структура реализуемого web-приложения — содержит в себе диаграмму и описание всех модулей, из которых состоит веб-приложение.

2.4. Описание сценариев взаимодействия с web-приложением — содержит в себе описание возможных вариантов использования приложения.

2.5. Фрагменты кода приложения/Описание используемых процедур и функций

3. ОРГАНИЗАЦИОННО-ЭКОНОМИЧЕСКАЯ ЧАСТЬ — раздел содержит в себе описание стоимости проекта и сопроводительную документацию к нему

3.1. Расчет затрат на разработку приложения — содержит теоретический расчёт стоимости разработки системы с учётом оплаты персонала, уплаты налогов, и прочих производственных расходов.

3.2. Эксплуатационный раздел/Руководство пользователя

3.3. Техника безопасности

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1Описание предметной области

В течение своего срока обучения, студентам высших и специальных учебных заведений приходится писать множество студенческих работ. Иногда цели написания этих студенческих работ могут быть довольно общими, и тогда тема работы может быть довольно свободной. Однако, для исключения возможности сдачи студентами чужих работ, должно выполняться условие уникальности тематики. При ведении бумажной документации, преподавательскому составу приходится самостоятельно следить за уникальностью тематик. Данная разработка намерена автоматизировать процесс поиска схожих работ, для сохранения уникальности тематик.

1.2Постановка задачи

Основания для разработки

Разработка ведётся на основании необходимости в автоматизации процессов систематизации и хранения тем студенческих работ. Так же приложение поможет следить за уникальностью работ.

Назначение разработки

Разработка предназначена для поиска существующих работ внутри дисциплины, что поможет пользователю отслеживать и соблюдать уникальность тем студенческих работ.

Описание требований к аппаратным и программным компонентам:

Для функционирования веб-приложения в системе необходимо наличие следующих компонентов:

- Компьютер с поддержкой python версии 3 и открытым сетевым портом 5000 - "сервер"
- Компьютер с веб-браузером - "клиент"

В общем случае, роль клиента и сервера может использовать один и тот же компьютер. В случае раздельной конфигурации, сервер и клиент должны находиться внутри одной локальной сети.

Выбор операционной системы для компьютера-сервера ограничен только фактом наличия на ней и корректного функционирования интерпретатора python и используемых библиотек. Данный проект планируется тестировать на корректность работы на операционных системах windows 10 и linux (debian 11). Так же проект может поставляться в виде открытого исходного кода, и гипотетически он должен работать на абсолютном большинстве актуальных современных операционных систем, в том числе mac OS и BSD.

Роль компьютера-клиента может исполнять как настольный персональный компьютер, так и мобильное устройство. Главным условием является наличие графической оболочки, и веб-браузера. Браузеру не обязательно уметь обрабатывать css и javascript, однако наличие возможности работы с ними значительно улучшат пользовательский опыт от работы с сайтом, и упростят навигацию по нему.

Определение протоколов связи:

Сервер предоставляет доступ клиенту посредством веб-интерфейса по протоколу HTTP по порту номер 5000.

Технические характеристики и функциональные возможности веб-приложения:

Веб-приложение должно позволять выполнять следующие функции:

- просматривать информацию в базе данных по всем таблицам
- производить неточный поиск по темам
- добавлять информацию в базу данных

Все перечисленные функции должны быть доступны конечному пользователю посредством простого интерфейса. В случае, если интуитивно понятный интерфейс разработать не удалось, или если вложение ресурсов разработки в данный аспект проекта будет признано нецелесообразным, то необходимо в обязательном порядке составить подробное руководство по пользованию разработанным интерфейсом.

Стадии и этапы разработки

Проект должен развиваться по следующей схеме:

1 Проектирование

На этапе проектирования должна быть сформирована архитектура модулей приложения

2 Разработка

На этапе разработки должны быть реализованы база данных и приложение для работы с ней

3 Тестирование

На этапе тестирования должны быть проведены проверки на соответствие системы требованиям, представленным в данном техническом задании, а также на устойчивость системы.

4 Внедрение и выпуск

На этапе внедрения система должна быть сформирована в поставочный пакет, удобный в установке конечным пользователем, и этот пакет должен быть передан пользователю.

5 Порядок контроля и приёмки

Программа должна функционировать согласно техническому заданию на системе конечного пользователя.

Методы тестирования и критерии успешности тестирования:

Тестирование приложения будет проходить по многоэтапной схеме: сначала будет проведено unit-тестирование (модульное тестирование) затем будут проводится интеграционные тесты готовых компонентов, и затем будут проведены тесты полными пользовательскими сценариями.

Unit-тестирование — оно же модульное тестирование — это набор низкоуровневых тестов наименьших модулей приложения. Суть юнит-тестирования заключается в написании тестов для каждого специфичного фрагмента кода (метода или функции). Это позволяет быстро проверить, что после внесения изменений в покрытый тестами фрагмент кода, его функциональные параметры не изменились, или что изменения не приведут к появлению ошибок далее по программе. Так же наличие юнит-тестов упрощает обнаружение и локализацию ошибок после проведения более высокоуровневых тестов.

Модульное тестирование проводится по методу белого ящика — тестировщику предоставляется вся информация о внутренней структуре

тестируемого модуля. В случае с данным этапом тестирования, этот метод упрощает последующее устранение ошибок поведения модуля.

Компонентное тестирование — это тестирование, целью которого является удостоверится в корректном функционировании раздельных модулей программы. В случае с данным проектом, как отдельный компонент будет протестирована база данных, используемая в приложении.

Данное тестирование проводится по методу серого ящика — при тестировании учитывается наличие некоторой информации о том, как необходимо пользоваться компонентом, и чего от него ожидать. Данное тестирование симулирует сценарий использования, когда пользователю была предоставлена инструкция/руководство по пользованию компонента.

Последним этапом проверки работоспособности приложения будет тестирование полными пользовательскими сценариями — т. е. после завершения разработки интерфейса к приложению, будут проведены проверки на функциональность интерфейса и правильность поведения приложения целиком.

Более низкоуровневые тесты (модульное и компонентное тестирование) разрабатывались согласно методу белого ящика — при составлении и проведении тестов учитывалась непосредственно внутренняя логика и устройство тестируемого элемента, его детали работы. Данный подход к тестированию, в случае проверки на корректность малого компонента, позволяет более подробно разобраться в логике тестируемого объекта, а так же упрощает последующее внесение исправлений в логику компонента.

В случае тестирования на взлом, метод белого ящика считается крайним и наиболее показательным на предмет безопасности системы.

Высокоуровневые, обобщённые тесты (тестирование системы пользовательскими сценариями, тестирование интерфейса) проводились по методу серого ящика. Метод серого ящика подразумевает, что у конечного

пользователя тестируемой системы имеются базовые необходимые знания о том, как пользоваться системой, однако отсутствуют знания о непосредственно внутреннем устройстве процессов и структур в системе. Данный метод позволяет больше углубиться в пользовательский опыт при пользовании системы, оценить простоту эксплуатации приложения, и доработать непосредственно инструменты взаимодействия с системой. В последствии минимальные знания по использованию приложения предоставляются посредством руководства пользователя.

В случае, если бы система предоставлялась без руководства пользователя, или поставлялась более широкому кругу пользователю, то возникла бы необходимость провести тестирование интерфейса и системы в целом по методу чёрного ящика. При проведении тестирования согласно методу чёрного ящика, считается, что о внутренней системе неизвестно ничего, и всю информацию необходимо получать непосредственно из приложения. Это наиболее показательный тест для проверки удобства пользования приложения и интуитивности понимания интерфейса. В идеале, данный вид тестирования проводится третьими лицами, чтобы быть уверенным в отсутствии знаний о структуре и поведении системы.

Задачи:

- проектирование общего плана приложения и его модулей: Спланировать структуру модулей приложения и логику взаимодействия между ними. Продумать поведение приложения. Утвердить ТЗ.
- Поэтапная разработка модулей приложения: Согласно проекту, разработать базу данных, веб-приложение и его интерфейс.
- Тестирование и отладка: Провести тестирование всех модулей и системы в целом, выявить и устранить обнаруженные ошибки и несоответствия функционала плану.

- Документирование проекта: Подготовить подробную документацию по проекту, включающую описание функционала, инструкции по установке и использованию веб-сервера, а также руководство по дальнейшему развитию проекта.

1.3Анализ существующих программных решений

Для выполнения постановленной задачи, учёт можно вести в базе данных, и взаимодействовать с ней через общее приложение для управления базами данных, например БД SQLite + приложение SQLiteStudio. Для реализации нечёткого поиска, в таком случае, можно будет использовать расширение полнотекстного поиска, например fts5 для SQLite. Однако, в случае с такой системой, от конечного пользователя потребуются знания обращения с языком SQL и СУБД.

Так же систему можно реализовать в формате Excel-книги. Данный подход требует другой, более распространённой специализации — навыкам работы с офисными продуктами. Однако, в случае с книгой сложнее будет оформить абстракцию для защиты данных. Поэтому данный вариант тоже нельзя считать полноценным.

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 Описание алгоритмов решаемых задач

В начале проектирования была разработана IDEF-0 диаграмма, описывающая в целом процесс работы приложения. Диаграмма приведена ниже на рисунке 1

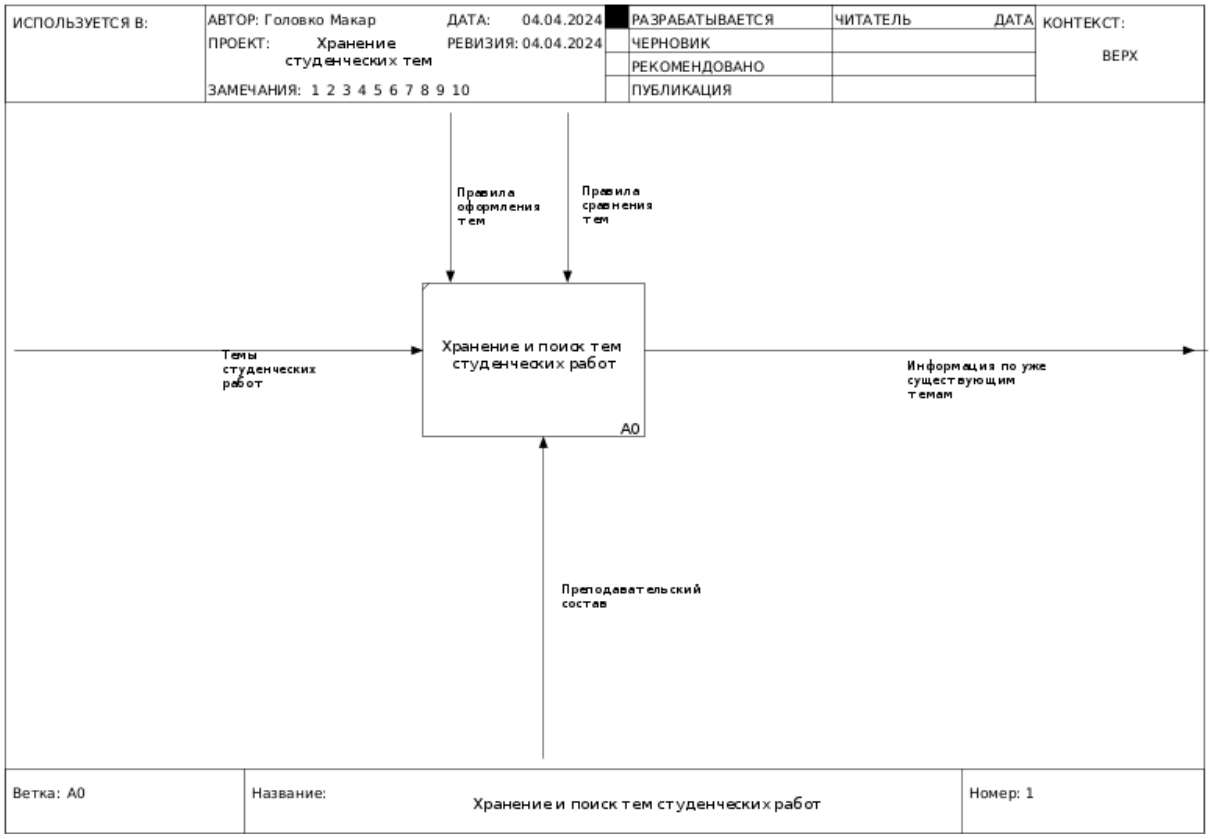


Рисунок 1 IDEF-0 диаграмма процесса работы приложения

В данной диаграмме определён основной процесс, обрабатываемый приложением — хранение и поиск тем студенческих работ. В процессе своей работы, приложение обрабатывает введённые в неё темы студенчесских курсовых и дипломных работ, и согласно полученной информации создаёт отчёты о схожих темах.

Входная информация в системе должна соответствовать условным правилам оформления тем. Данное условие будет соблюдаться засчёт разбития темы на вид и содержание работы. Затем система сравнивает элементы введённых тем с информацией уже имеющейся в системе. Сравнение идёт с помощью расчёта дельты редактирования между элементами, и затем выводятся варианты с индексом схожести выше определённого параметра.

Субъектами системы и конечными пользователями приложения будут работники образовательной организации, в частности преподавательский состав. Систему можно настроить для общего пользования внутри одной локальной сети.

2.2Обоснование выбора инструментов разработки

Согласно требованиям к программе, для разработки приложения были выбраны следующие инструменты разработки: язык программирования Python, микро веб-фреймвор Flask, система управления базами данных SQlite 3, коммандная оболочка Bash, текстовый редактор NeoVim, инструмент администрирования и редактирования баз данных SQLite Studio, инструмент для рисования диаграмм Draw.io.

Для разработки всего исходного кода приложения использовалась командная оболочка bash и текстовый редактор NeoVim. Выбор в пользу работы внутри командной строки и NVim-а был сделан на основании быстродействия подобной среды разработки. Так же консоль позволяет более тонко управлять запуском и отладкой приложений.

Vim представляет собой бесплатную программу с открытым исходным кодом, предназначенную для работы с текстом. Vim является

усовершенствованным клоном старого текстового редактора Vi. Особенностью редактора является нестандартная структура работы в нём, наличие различных режимов работы в нём и управление через команды. Помимо непосредственно режима ввода текста, иные задачи редактирования текста и навигации по нему проводятся в "нормальном" режиме. В этом режиме нельзя вводить текст, и клавиатура используется для выполнения операций с текстом. Данный дизайн позволяет, при должной сноровке, вести редактирование и написание текста с помощью меньшего кол-ва манипуляций. Так же данное управление посредством команд помогает оформить автоматизацию редактирования элементов текста.

Помимо функционала "из коробки", Vim поддерживает тонкую настройку своего поведения, а так же пользовательские расширения — плагины. Главным отличием NeoVim от класического Vim-а является более широкая поддержка пользовательских расширений. Посредством данных расширений в NeoVim можно добавить мощный функционал из более привычных IDE (таких как Visual Studio и продукты JetBrains), такой как колорайзеры и автозаполнение, при этом не теряя в производительности и быстродействии системы и редактора.

Язык программирования Python был выбран на основе следующих причин:

1 Простота разработки. Синтаксис Python близок к естественным языкам, что позволяет писать очень удобочитаемый и простой код.

2 Кросплатформенность. Python позволяет развернуть приложение на различных платформах – Windows, Mac, Linux. Это упрощает требования к оборудованию предприятия.

3 Модульность. Для языка Python сообществом было разработано множество различных библиотек, что позволяет не разрабатывать самому модули с нуля.

Исходя из этих факторов, Python был выбран как основной язык программирования для данного проекта.

Сервер приложения предоставляется модулем flask. Flask — это микро веб-фреймворк, написанный на языке программирования python. Он классифицирован как микро-фреймворк, потому что для работы с ним не требуется сторонние библиотеки и инструменты. В нём нет встроенных слоя абстракции базы данных и валидации форм. Это позволяет разрабатывать максимально простые минимальные веб-приложения. Однако, по усмотрению разработчика, можно воспользоваться разработанными сообществом библиотеками (SQAlchemy и WTForns соответственно). Однако в силу уже изученного набора технологий и короткого срока, выделенного на изучение новых инструментов, был сделан выбор в пользу SQLite.

Flask содержит в себе и использует следующие компоненты:

- Werkzeug (нем. "Инструмент") — библиотека-утилита на Python для работы согласно стандарту WSGI (Web Server Gateway Interface). В общем случае, именно Werkzeug отвечает за инстансирование объектов для запросов, и работу по http протоколу в целом.
- Jinja — движок шаблонов веб-страниц. Jinja схожа с языком шаблонов для Django, однако своим синтаксисом используемый в данном проекте движок больше напоминает синтаксис python. Данный движок позволяет кастомизировать теги, производить операции над переменными, проводить условные сравнения, и поддерживает глобальные переменные. Так же, Jinja поддерживает вызов функций. Этот движок шаблонов так же поддерживает следующий функционал:

запуск файла в sandbox-режиме; автоматическая html-"санитизация" (функционал для обеспечения безопасности сервера, очищающий информацию, полученную от пользователя, не позволяющий запускать кастомные скрипты на сервере приложения); Наследование шаблонов; удобная отладка.

- MarkupSafe — это библиотека работы со строками для языка программирования Python. Собственный тип MarkupSafe расширяет встроенный в Python тип string, и проверяет его объекты на безопасность. Объединение типов Sting и MarkupSafe автоматически "закрывает" (ставит escape-символ в конец) не помеченные строки, и при этом избегает двойное закрытие уже помеченных строк.
- ItsDangerious — библиотека безопасной сериализации информации для языка Python. Позволяет Flask-приложению сохранять рабочую сессию в cookie-файле, при этом не позволяет пользователю его редактировать. С помощью сессии у приложения появляется возможность переносить пользовательскую информацию с одной страницы на другую (например, в ситуации поиска).

Интерфейс приложения был разработан с помощью веб-технологий. Выбор данного модуля был обоснован следующими факторами:

1 Веб-формат. Flask реализует сервер, который предоставляет адаптивные генерируемые web-страницы, и обрабатывает веб-формы get и post. Такой формат позволяет организовать запуск приложения на едином сервере с удалённым доступом на него, но всё ещё допускает локальную работу с приложением.

2 Универсальность. Как было сказано ранее, за счёт веб-формата приложение работает по клиент-серверной структуре. Это позволяет настроить систему для общего пользования внутри

одной локальной сети, или же развернуть сервер и клиент на одном и том же компьютере.

3 Широкие возможности в разработке интерфейса. Веб-стек позволяет разрабатывать самые разнообразные виды интерфейса и формы для взаимодействия с пользователем.

Помимо html и css, в разработке используется фреймворк bootstrap. Bootstrap представляет собой набор css и javascript скриптов, использование которых упрощает разработку сайта, и реализует некоторый специфичный функционал, а так же позволяет разрабатывать более красивые веб-страницы. С его помощью оформлена более красивая, по сравнению с стандартным html, шапка сайта.

Для хранения информации был выбран формат SQLite с использованием соответствующей библиотеки sqlite3 для python. Выбор был сделан на основании того, что SQLite можно легко интегрировать непосредственно в приложение. БД на SQLite представляет собой простой файл, а СУБД содержится в разрабатываемом приложении, и в итоге от конечного пользователя системы не требуется установка, настройка и поддержания отдельного серверного приложения для её обслуживания. Это многократно упрощает развёртку приложения для конечного пользователя.

Для разработки базы данных использовался консольный инструмент системы управления базами данных sqlite3 и графическое приложение SQLite Studio. За счёт работы через консольную оболочку, можно подробно записать запросы на генерацию БД и добавление тестовых данных. Далее программу можно будет поставлять уже с документами, содержащими эти отчёты. SQLite Studio использовался для упрощения просмотра структуры БД и записей в ней.

Для сборки приложения в единый запускаемый файл использовался инструмент pyinstaller. Данный модуль для python позволяет организовывать python-скрипты и все их зависимости в единый файл для удобной поставки конечному пользователю. Оффициально pyinstaller поддерживает сборку под windows, linux и mac OS, однако возможна работа и с другими операционными системами (BSD, Solaris)

Для разработки диаграмм к документации использовалась платформа DrawIO (теперь называемый diagrams.net). Этот инструмент представляет собой кросс-платформенное приложение для составления графики и диаграмм, разработанное на HTML5 и JavaScript. Набор компонентов в Draw.io позволяет составлять самые различные виды диаграмм, такие как Use-case, блок-схемы, инфологические и даталогические диаграммы для описания устройства баз данных, диаграммы потоков данных, и так далее. Diagrams.net доступен в виде веб-приложения, однако в разработке данного дипломного проекта использовалась локальная версия для операционной системы linux.

2.3Структура реализуемого web-приложения

Диаграмма модулей веб-приложения приведена на рисунке 2:

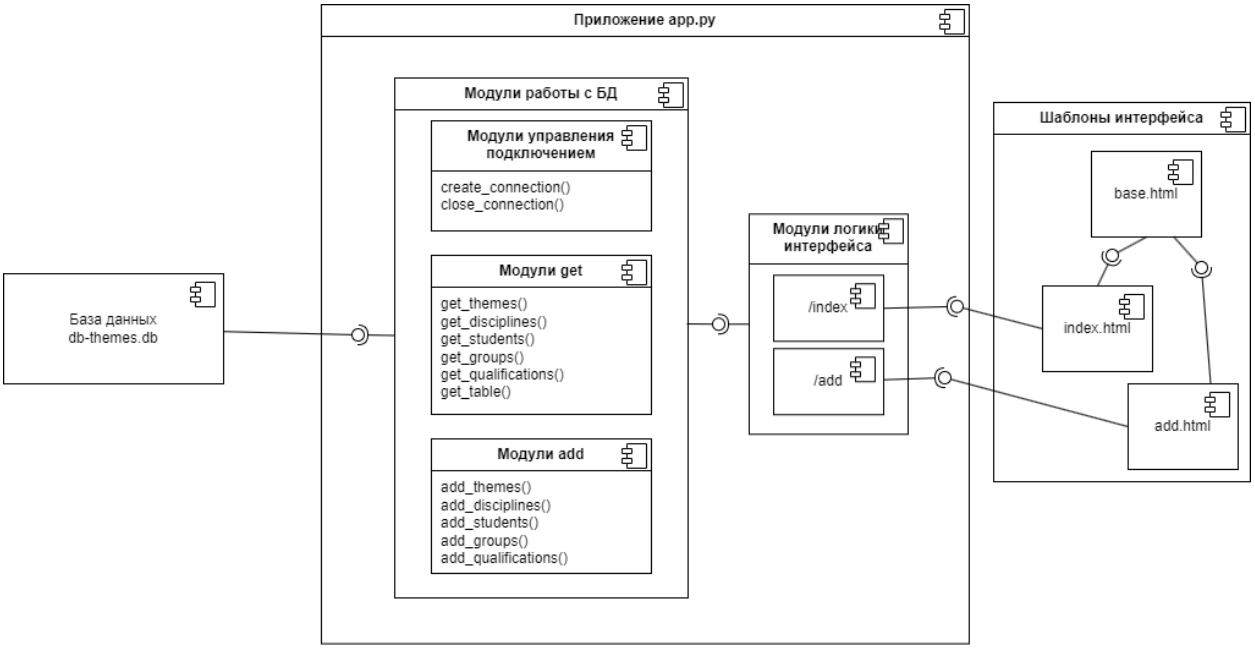


Рисунок 2 Диаграмма модулей программы

В диаграмме определено 3 основных элемента приложения: база данных, собственно приложение-сервер, и пользовательский веб-интерфейс.

База данных представляет собой набор таблиц, записанных в файл по формату SQLite. Структура базы данных, используемой приложением, приведена на рисунке 3:

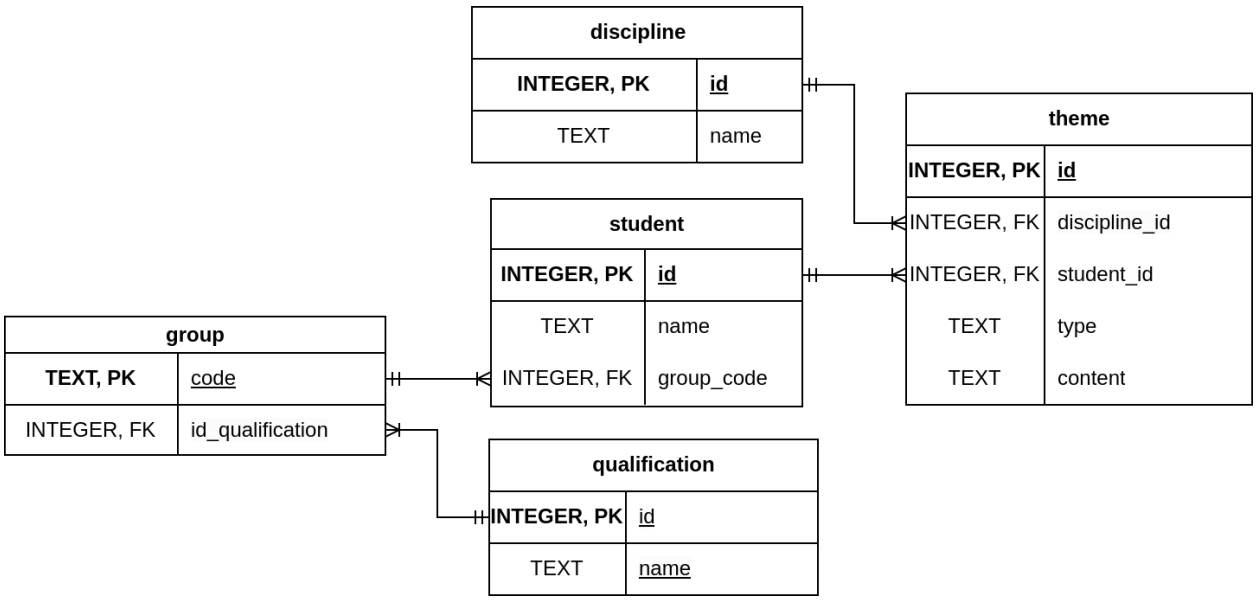


Рисунок 3 даталогическая диаграмма базы данных

Данная база данных впоследствии используется python-приложением (app.py), а именно модулями работы с БД. Данные модули используют для своей работы библиотеку sqlite3. Эти модули используются в модуле обработки логики интерфейса, который заполняет html-шаблоны и обрабатывает get-post запросы посредством библиотеки flask. Так же в логике интерфейса используется функция нечёткого поиска, реализованная посредством библиотеки difflib.

В данной базе данных определены следующие таблицы:

- Theme — Тема студенческой работы, основная таблица. В ней хранится непосредственно тема, разбитая на её вид и её содержание, идентификатор дисциплины, к которой относится данная работа, и идентификатор студента, который её написал. Например, в теме "Разработка веб-страницы для аптеки", вид работы будет "разработка веб страницы", а содержание — "аптека". Данное разделение сделано для упрощения организации поиска по системе.
- Discipline — Дисциплина, т. е. "предмет", к которому относится студенческая работа. Например, "Инструментальные средства разработки программного обеспечения", или "Участие в интеграции программных модулей".
- Student — Студент, база данных хранит ФИО студента и идентификатор группы, к которой он относится. Так же, у каждого студента есть свой идентификатор, так что система корректно ведёт себя с полными тёсками и переведёнными между группами студентами.
- Group — Группа, хранит в себе код группы и идентификатор квалификации, к которой она относится. Например, для группы П1-20 код будет "П1-20", а квалификация будет "Техник-программист".

Группы П2-20 и П3-20 так же должны будут быть внесены как группы с такой квалификацией.

- Qualification — Квалификация, обобщающий параметр для студенческих групп. Группы ИС1-20 и ИС2-20 имеют различные коды групп, но должны быть записаны как относящиеся к одной квалификации — "Информационные системы".

Столбцы таблиц базы данных подробно описаны в таблицах 1-5

Таблица 1

Таблица "theme"

Наименование столбца	Тип данных	Описание	Ограничения
Id	INTEGER	Уникальный идентификатор	Primary key
discipline_id	INTEGER	Идентификатор релевантной дисциплины	Foreign key
student_id	INTEGER	Идентификатор студента - автора работы	Foreign key
type	TEXT	Тип работы ("Веб-приложение", или "База данных")	
content	TEXT	Содержание, тематика работы (например, "Магазин", или "Депо")	

Таблица 2

Таблица "discipline"

Наименование столбца	Тип данных	Описание	Ограничения
id	INTEGER	Уникальный идентификатор	Primary key
name	TEXT	Наименование дисциплины	

Таблица 3

Таблица "student"

Наименование столбца	Тип данных	Описание	Ограничения
id	INTEGER	Уникальный идентификатор	Primary key
name	TEXT	Фамилия, имя и отчество студента	
group_code	INTEGER	Код группы, в которой учится студент	Foreign key

Таблица 4

Таблица "group"

Наименование столбца	Тип данных	Описание	Ограничения
code	TEXT	Уникальный идентификатор	Primary key
qualification_id	INTEGER	Идентификатор релевантной квалификации	Foreign key

Таблица 5

Таблица "qualification"

Наименование столбца	Тип данных	Описание	Ограничения
id	INTEGER	Уникальный идентификатор	Primary key
name	TEXT	Наименование квалификации	

Непосредственно приложение можно условно поделить на две части — модуль работы с базой данных, и модуль, отвечающий за логику веб-интерфейса приложения.

Модуль работы с базой данных содержит в себе всю необходимую абстракцию для упрощения последующей работы. Он отвечает за подключение/отключение от базы данных, а так же реализует специфичные методы для получения информации из бд и добавления записей в неё.

В данном модуле определены следующие методы:

- create_connection() — метод подключения к БД. Использует фиксированные параметры для подключения или создания базы данных. Так же обрабатывает ошибки подключения. Возвращает специальный объект подключения к базе данных приложения.
- close_connection() — метод безопасного отключения от базы данных. Возвращает информацию об успешности завершения своей работы, иначе обрабатывает ошибку.
- Методы Get — набор методов, который возвращает заголовок запрошеной таблицы (список, состоящий из имён таблиц на

английском языке), и список с непосредственно всеми строками из запрошеной таблицы. Засчёт возвращения заголовка таблицы, в последующем приложению гораздо проще составить понятный интерфейс для чтения таблицы.

- Методы add — наобор методов, каждый из которых берёт на ввод специфичный набор аргументов, соответствующий требуемой таблицы.

Модуль логики интерфейса отвечает непосредственно за веб-сервер и обработку http запросов. Т.е. в этой части приложения прописано, какая информация отправляется в шаблоны, и как обрабатывается информация, получаемая из форм, заполняемых пользователем. В ней отдельно обрабатывается логика страницы отображения информации из базы данных и страницы добавления данных. Так же на главной странице обрабатывается форма поиска тем. По своей сути, именно серверная часть приложения заполняет html-шаблоны с помощью встроенного в flask движка jinja2.

Сами документы веб-страниц представляют собой шаблоны,н унификации, имеется общий шаблон, в котором определён html-скелет, а так же общая шапка для переключения между страницами. В последствии, страницы наследуют этот шаблон, и заполняют его своими элементами. В самих шаблонах, посредством циклов отрисовывается информация, отправленная сервером. В случае, когда требуется запрос информации от пользователя, то используется элемент разметки <form>, благодаря которому сервер понимает, откуда собирать данные.

2.4Описание сценариев взаимодействия с web-приложением

Взаимодействие с приложением может быть описано с помощью диаграммы вариантов использования, приведённой на рисунке 4:

Рисунок 4 Диаграмма вариантов использования приложения

На данной диаграмме приведены следующие варианты использования приложения:

1 Добавление информации в систему. Подразумевает собой возможность пользователю добавить информацию в базу данных для постоянного хранения и обработке при поиске. Т.е. у системы должна быть страница интерфейса с достаточным функционалом для того, чтобы пользователь мог корректно ввести верную информацию в систему. Данная информация в последствии будет использоваться при поиске похожих тематик студенческих работ. Для остальных таблиц, при вводе информации, у пользователя должна быть возможность установить правильные связи между элементами таблиц (т. е. связи

студент - его группа - её квалификация). Т.е. система должна помогать пользователю вносить корректную структуру.

2 Получение информации из системы. Обобщённая группа из нескольких вариантов использования. Данный вариант подразумевает под собой то, что у пользователя должен быть понятный и информативный интерфейс для получения данных из системы.

2.1 Просмотр таблиц. Интерфейс приложения должен позволять пользователю просмотреть всю информацию, хранящуюся в системе , в удобном виде. Это так же значит, что все таблицы должны быть корректно подписаны.

2.2 Получение похожих тем студенческих работ. У пользователя должна быть возможность посредством интерфейса программы запросить некоторую тему студенческой работы, или кандидата на неё, в ответ на что система должна вернуть похожие темы студенческих работ из списка тех, что хранятся в системе. Данная функция является ключевой в системе.

Взаимодействие с приложением идёт по следующему алгоритму: пользователь запускает приложение, открывает веб-страницу приложения, перед пользователем открывается страница по умолчанию — страница вывода информации с открытой таблицей тем. На ней пользователь может выбрать необходимую таблицу для отображения, или перейти на страницу для ввода информации. При переходе на страницу для ввода информации перед пользователем открывается пустая таблица для заполнения с подписанными колонками. На этой странице пользователь может так же выбрать другую таблицу для ввода информации. После заполнения необходимой информации пользователь подтверждает ввод данных кнопкой "добавить".

Более подробно алгоритм работы с приложением описан в главе 3.2 Руководство пользователя.

2.5Описание используемых процедур и функций

В данном разделе будут более подробно описаны функции логики интерфейса, написанные в сервере. Для просмотра исходного кода, обратитесь к приложению к данной записке к дипломному проекту

В листинге 2 представлен код непосредственно веб-приложения/сервера. В первой его части написан набор функций для работы с базой данных. Он был описан в главе 2.3 Структура реализуемого веб-приложения. После него, в главной части приложения написана непосредственно логика приложения. Данный код представляет из себя функции двух страниц — главной страницы (адрес "/" или "/index/") и страницы добавления информации (адрес "/add/").

Главная страница работает по следующему алгоритму: Сначала создаётся подключение с базой данных, затем страница уточняет, какую именно страницу сейчас нужно отобразить. Данная информация передаётся на сервер в виде http-аргумента, который вписывается в адрес страницы. Например, для просмотра таблицы студентов, ссылка будет дополнена параметром table со значением students: "/?table=students". Если в сервер не передали данного параметра, то по умолчанию отображается таблица тем студенческих работ. Само отображение таблицы работает довольно просто — сервер запрашивает заголовки столбцов таблицы и само её содержимое из базы данных через обращение к модулю управления БД, и затем передаёт эту информацию на отрисовку. Вторая часть логики страницы обрабатывает запросы на поиск в системе. Сервер получает информацию из формы

запроса, выполняет поиск с помощью внешней функции, и возвращает результат на страницу.

Страница добавления так же работает довольно просто за счёт написанного ранее модуля работы с БД. Данной функции остаётся только связать этот модуль с интерфейсом и заполнить шаблон страницы. Функция получает http-параметр, по той же логике, что и на главной странице, запрашивает подходящий набор заголовков, и уже за счёт них заполняет шаблон с формой для пользовательского ввода.

Переадресация между страницами и их непосредственное внутреннее устройство описаны внутри html-шаблонов.

3 ОРГАНИЗАЦИОННО-ЭКОНОМИЧЕСКАЯ ЧАСТЬ

3.1Расчёт стоимости разработки приложения

Одним из основных экономических показателей любого продукта является стоимость его создания (разработки). Для расчета стоимости разработки реализуемого проекта необходимо будет определить:

- временные затраты на реализацию каждого этапа работы;
- количество специалистов, задействованных в реализации проекта, и величину оплаты их труда;
- налоговые отчисления;
- затраты на амортизацию оборудования, на электроэнергию, на материалы и прочие затраты.

В таблице 6 приведены временные затраты на реализацию каждого этапа работы.

Таблица 6

Затраты времени по стадиям разработки *приложения*

№	Наименование стадии/этапа работ	Трудоёмкость, норма-час
1	Формирование требований к приложению	8
2	Разработка концепции	16
3	Техническое задание	24
4	Проектирование и разработка	80
5	Рабочая документация	32
ИТОГО за разработку:		160 часов

Общая трудоемкость составляет 160 нормо-часов.

Плановый срок разработки информационной системы (модуля информационной системы, приложения) – 4 недели, среднее количество рабочих дней в указанный период – 20, продолжительность рабочего дня – 8 часов. Тогда по плану срок работы 20*8 = 160 часов.

В таблице 7 указаны специалисты, задействованные в разработке приложения

Таблица 7.

Задействованные специалисты

№ п/п	Должность	Кол-во сотрудников	Оклад
1	Специалист по разработке	1	250 руб/час
ИТОГО за месяц, руб.:			40000

Определение себестоимости разработки

Себестоимость разрабатываемого приложения складывается из текущих затрат.

Текущие затраты включают в себя: зарплату и налоговые отчисления, амортизацию, затраты на электроэнергию, материалы, затраты на ремонт и запасные части (при необходимости), накладные и прочие расходы.

Затраты на оплату труда работников, непосредственно занятых разработкой приложения, определяются с использованием данных из таблицы 2 и составляют за срок разработки 40000 руб.

Затраты на отчисления по налогам:

Пенсионный фонд РФ (22%):

$$отчисления = сумма\ за\ срок\ разработки \times 22\% \quad (1)$$

40000 *0,22= 8800 руб.

Фонд социального страхования (2,9%):

$$отчисления = сумма\ за\ срок\ разработки \times 2{,}9\,\% \qquad (2)$$

40000 *0,029= 1160 руб.

Фонд обязательного медицинского страхования (5,1%):

$$отчисления = сумма\ за\ срок\ разработки \times 5{,}1\,\% \qquad (3)$$

40000 *0,051= 2040 руб.

Общая сумма затрат на отчисления по налогам составит:

$$\sum отчисление_{ПФРФ} + отчисление_{ФСС} + отчисление_{ФОМС} \qquad (4)$$

(1) + (2) + (3) = 12000 руб.

Затраты на амортизацию основных средств представлены в таблице 3 и рассчитываются по формуле 8.

$$амортизация = (общая\ стоимость \div срок\ службы) \times срок\ разработки \quad (5)$$

Таблица 8.

Затраты на амортизацию

Наименование	Модель	Цена, руб.	Кол-во	Общая стоимость	Срок службы мес.	Амортизация за 1 мес., руб.
Ноутбук	Packard bell easy note LV11HC-B9704G50MNKS	21 764	1	21 764	144	152
Кресло	RT007A	13500	1	13500	60	225
Лакированный стол	-	8000	1	8000	120	67
МФУ	HP LaserJet Pro M1212nf MFP	18400	1	184000	120	154

Амортизация по всем элементам за 1 месяц составит:

152 + 225 + 67 + 154 = 598 руб.

Затраты на электроэнергию рассчитываются следующим образом (формула 6):

$$затраты = количество \times мощность \times время\ рабочего\ дня\ х\ на\ количество\ дней\ разработки \quad (6)$$

рабочее помещение освещает одна LED-панель мощностью 36 Вт/час и настольная лампа мощностью 20 Вт/час (используется не весь день), в день используется: 36*8+20*4= 368Вт, за весь период разработки 368*20= 7360 Вт

потребляемая мощность компьютера составляет 90 Вт/ч, в день используется: 90*8= 720Вт, за весь период разработки 720*20= 14400 Вт

При расчёте стоимости киловват-часа при одноставочном тарифе в 5,66 руб./кВт-ч затраты на электроэнергию за 1 месяц составят 42+ 82= 124 руб.

Затраты на материалы, используемые при разработке приложения представлены в таблице 9.

Таблица 9.

Затраты на материалы

№ п/п	Наименование материала	Единицы измерения	Количество, шт.	Цена, руб.	Общая стоимость, руб.
1	Бумага	уп.	1	200	200
ИТОГО					200

Прочие затраты составляют 3000 руб.

Суммарные затраты представлены в таблице 10.

Таблица 10

Суммарные затраты

Статья затрат	Сумма, руб.
Заработная плата	40000
Отчисления по налогам	12000
Амортизация	598
Электроэнергия	124
Материалы	200
Прочие	3000
ИТОГО	55922

Таким образом затраты на разработку приложения составили 55922 руб.

3.2 Эксплуатационный раздел/Руководство пользователя

для работы с приложением, сначала необходимо запустить сервер. Для этого, пользователю необходимо запустить приложение

Алгоритм работы с веб-интерфейсом может быть описан посредством блок-схемы на рисунке 5:

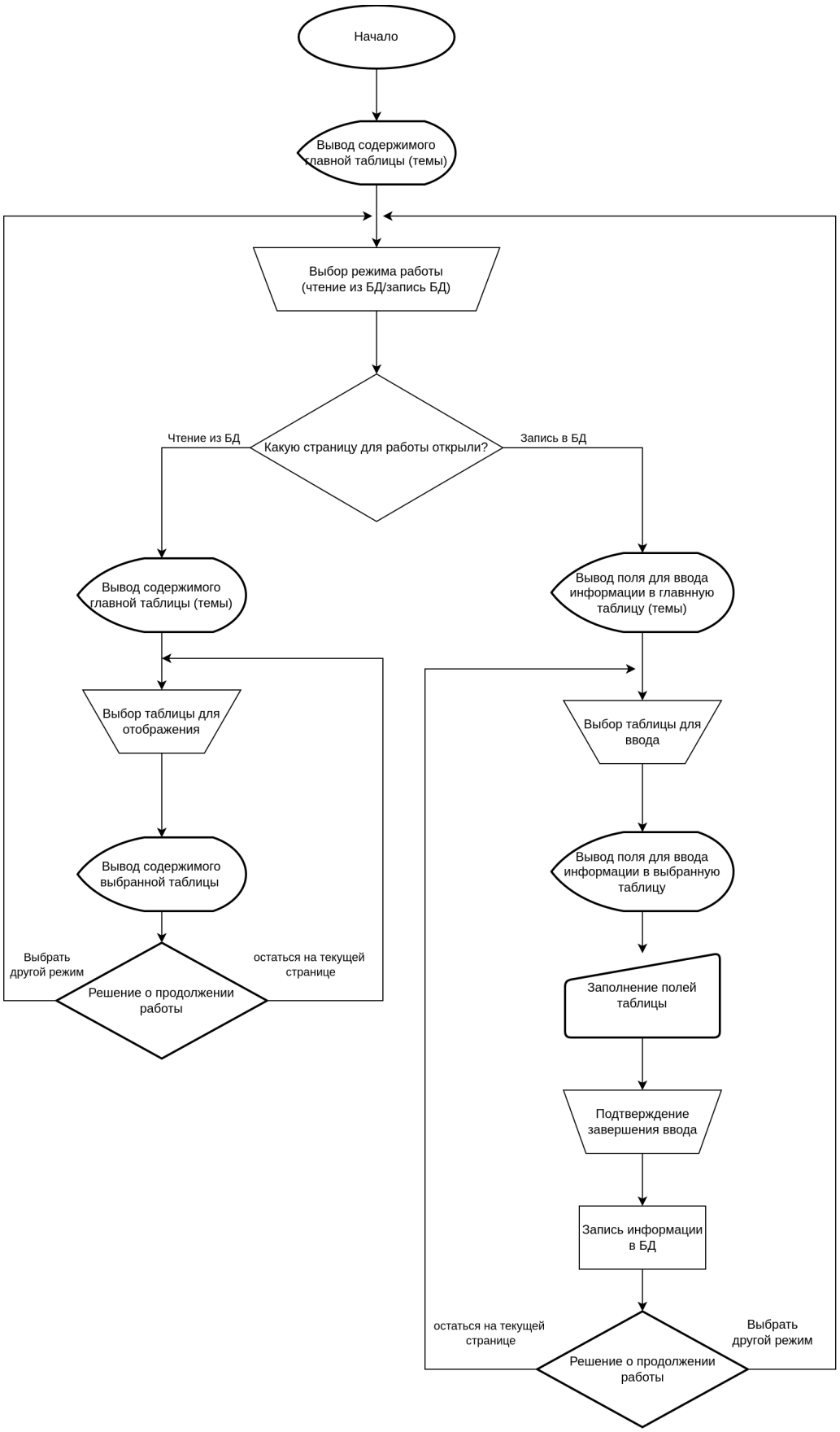
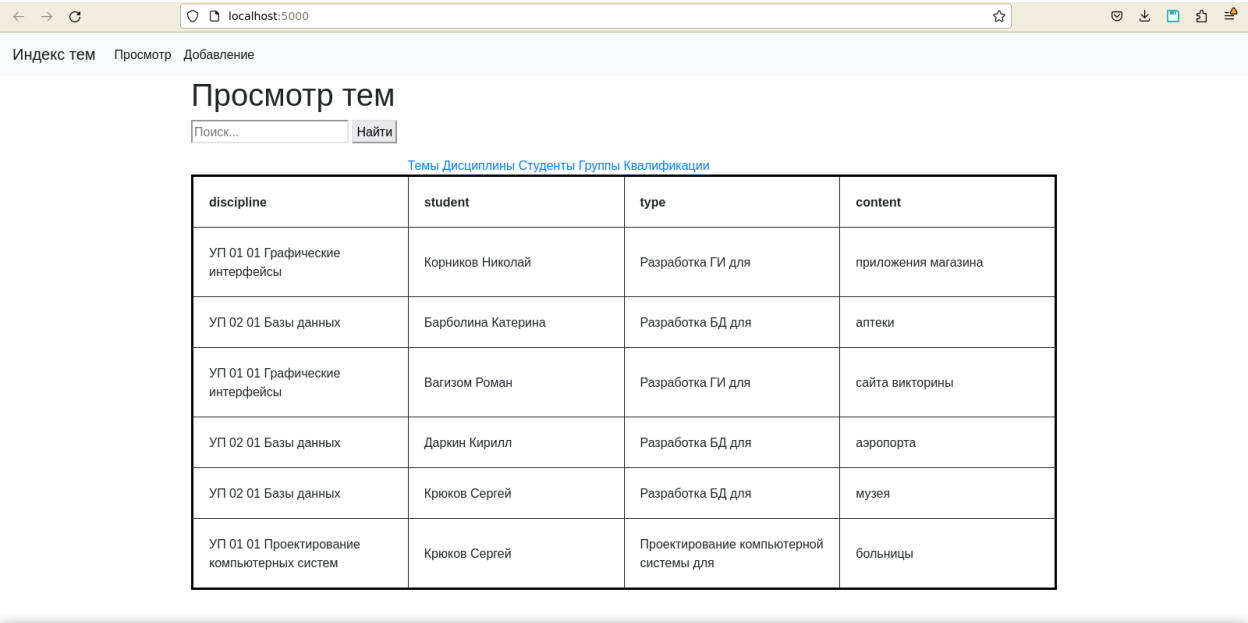


Рисунок 5 Блок-схема работы с веб-интерфейсом

Алгоритм на данной диаграмме можно описать следующим образом: Пользователь открывает веб-приложение, ему выводится главная страница с отображением содержимого таблиц тем, пользователю даётся возможность начать эксплуатацию интерфейса. Пользоваетль может выбрать — остаться ли ему на данной странице, или, посредством ссылок в шапке, перейти на страницу добавления. Если пользователь остаётся на главной странице (странице вывода), то он может выбрать другие таблицы для просмотра. Тогда сайт выведет пользователю запрошенную таблицу, и он сможет просмотреть её. После этого, пользователь может свободно выбрать любую другую таблицу для просмотра, или перейти на страницу добавления.

Если пользователь переходит на страницу добавления, то алгоритм работы с ней довольно схож. Пользователь выбирает таблицу для ввода, заполняет её и отправляет на сервер. Засчёт одинакогово интерфейса "шапки" сайта, пользователь может легко перейти обратно на страницу просмотра.

При открытии веб-адреса работы приложения (при локальной работе: localhost:5000), перед пользователем открывается страница чтения информации. Интерфейс страницы чтения информации предоставлен на рисунке 6:



discipline	student	type	content
УП 01 01 Графические интерфейсы	Корников Николай	Разработка ГИ для	приложения магазина
УП 02 01 Базы данных	Барболина Катерина	Разработка БД для	аптеки
УП 01 01 Графические интерфейсы	Вагизом Роман	Разработка ГИ для	сайта викторины
УП 02 01 Базы данных	Даркин Кирилл	Разработка БД для	аэропорта
УП 02 01 Базы данных	Крюков Сергей	Разработка БД для	музея
УП 01 01 Проектирование компьютерных систем	Крюков Сергей	Проектирование компьютерной системы для	больницы

Рисунок 6 Главная веб-страница

На данной странице пользователь может просмотреть содержимое всех таблиц. В форме отображаются подписи колонок, и, соответственно, информация из таблицы. Прямо над табличным отображением находятся вкладки для перехода к просмотру других таблиц. Пользователь может нажать на эти ссылки, и интерфейс переключится на отображение соответствующей таблицы.

Чуть выше находится окно для поиска схожих тем. Пользователь вводит название темы, которую он хочет найти, или которую он планирует добавить, и система пытается найти похожие по написанию темы. На основании этой информации пользователь принимает решение о изменении написания темы, или о изменении её содержания.

Через шапку сайта пользователь может перейти на страницу добавления информации. Интерфейс страницы добавления информации представлен на рисунке 7:

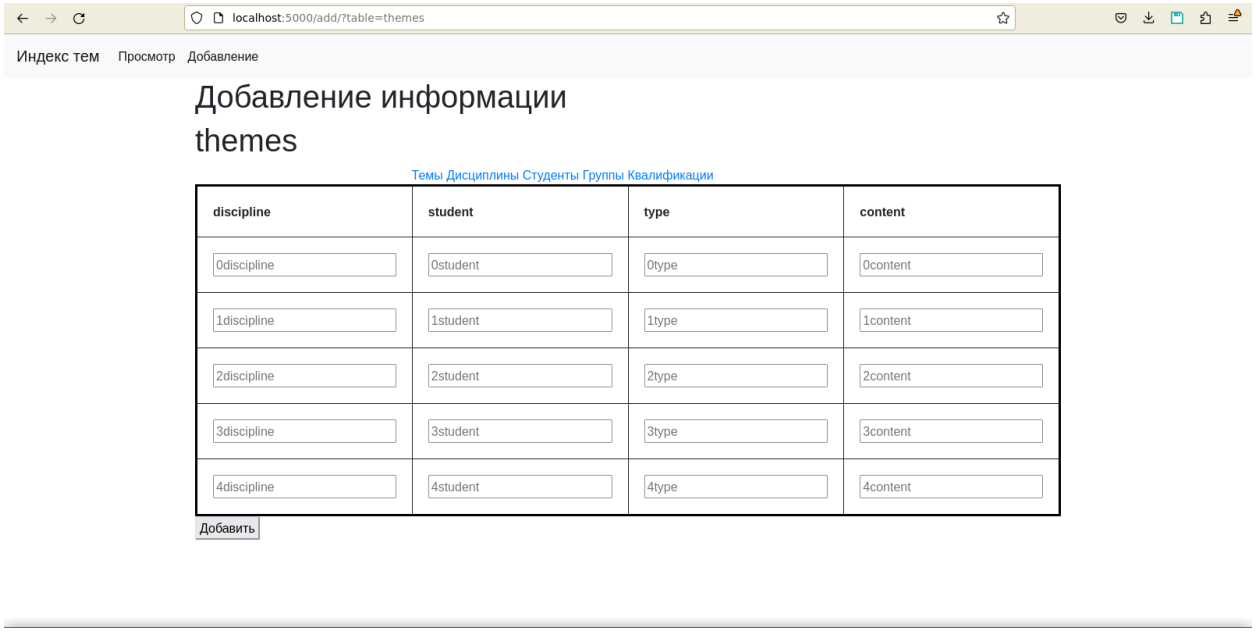


Рисунок 7 Страница добавления информации

На данной странице отображается шаблон под выбранную пользователем таблицу. Пользователь заполняет текстовые поля, и жмёт кнопку "Добавить", после чего форма очищается и информация записывается в базу данных.

3.3Техника безопасности

Использование персонального компьютера совсем не так безопасно, как можно подумать. Это убедительно подтверждается информацией, содержащейся в стандарте ГОСТ 12.0.003-2015, описывающем основные вредные и опасные факторы такого трудового процесса. В их список входят:

- высокая температура, характерная для отдельных элементов компьютерной техники и создающая общий повышенный температурный фон в рабочем помещении;
- высокая степень монотонности рабочего процесса;
- значительный уровень зрительный нагрузок, испытываемых работником;
- риск поражения статическим электричеством при случайном контакте с отдельными элементами техники;
- недостаточная освещенность рабочей зоны;
- повышенный уровень напряженности электрического и магнитного полей, генерируемых работающей техникой;
- высокий уровень блесткости и контрастности рабочего экрана, негативно влияющий на зрение работника в длительной перспективе;
- другие факторы.

Гигиенические требования

В связи с наличием воздействия большого списка вредоносных факторов на работника, требования техники безопасности при работе с компьютером охватывают не только сам процесс труда, но и сопровождающие его обстоятельства, которые также влияют на

работоспособность и сохранность здоровья сотрудника на длинных временных горизонтах. Соответствующие правила организации рабочего процесса приведены в СанПиН 2.2.2/2.4.1340-03. Они включают ряд нормативов, которые касаются следующих моментов:

- общие правила организации работы с использованием компьютерной и офисной техники;
- требования к персональному компьютеру, используемому для постоянной работы сотрудника;
- требования к помещениям, в которых выполняется работа с применением офисной техники;
- требования к микроклимату для соответствующих рабочих мест, включая содержание в воздухе рабочей зоны аэроионов и вредных химикатов;
- допустимый уровень шума, генерируемого рабочим оборудованием;
- правила организации освещения рабочей зоны;
- разрешенные нормативы в отношении параметров электромагнитных полей на рабочих местах;
- техника безопасности при пользовании компьютером, которая связана с организацией рабочих мест сотрудников;
- медицинский контроль за здоровьем персонала;
- порядок организации государственного санитарно-эпидемиологического надзора и выполнения производственного контроля.

Инструктаж по технике безопасности за компьютером

Чтобы донести до работника требования, которые устанавливает техника безопасности при использовании компьютера, работодатель обязан организовать инструктаж. Он проводится на основании специально составленной инструкции, которую обычно разрабатывает специалист по охране труда или непосредственный руководитель работника. Проведение инструктажа нужно организовать до начала самостоятельной работы: специалист допускается к работе только после его прохождения. Это значит, до него донесены правила техники безопасности при работе с компьютером. Инструкция-памятка по технике безопасности за компьютером должна быть выдана каждому работнику в бумажном или электронном виде. Ей можно дать вид легкочитаемой инфографики – например, 10 правил техники безопасности за компьютером.

Меры безопасности перед началом работы

В рамках проведения инструктажа до работника доводятся сведения о необходимых действиях, выполняемых на каждом этапе рабочего процесса. Техника безопасности в начале работы на компьютере требует выполнения следующих операций:

- проверить исправность элементов электросистемы, обеспечивающей питание компьютера, включая электропроводку, выключатели, вилки и розетки, при помощи которых аппаратура подключается к сети;
- проконтролировать заземление компьютера;
- проверить его работоспособность.

Техника безопасности возле компьютера в процессе работы

При выполнении работ нужно соблюдать следующие правила:

- запрещается класть на корпус и дисплей компьютера посторонние предметы, прикасаться к элементам аппаратуры мокрыми руками,

производить чистку корпуса оборудования, находящегося под напряжением, располагать технику близко к жилищно-коммунальным инженерным системам;

- в случае обнаружения неисправности компьютера немедленно прекратить работу и сообщить об этом непосредственному руководителю;
- эксплуатировать компьютер только с соблюдением инструкции, установленной производителем;
- избегать частого и необоснованного включения и выключения компьютера во время работы.

Техника безопасности по окончанию работы за компьютером

После завершения работы сотруднику нужно выполнить следующие действия:

- выключить компьютер с использованием алгоритма, установленного производителем;
- обесточить периферийное оборудование;
- убедиться в отключении техники;
- выполнить очистку рабочих поверхностей влажной тканью.

Правильное расположение за компьютером

Требования к расположению работника за компьютером нацелены на обеспечение его комфорта в течение всей рабочей смены и отсутствие негативных следствий длительной работы. Они действуют для любых рабочих мест, будь то бухгалтерия, обучающий класс или кабинет информатики. Они включают следующие правила:

- полная опора ступнями на пол при посадке;

- использование компьютерной мебели, отвечающей нормам СанПиН 2.2.2/2.4.1340-03;
- отказ от скрещивания конечностей, способного затруднить кровообращение;
- соблюдение расстояние до монитора компьютера не меньше 45 сантиметров;
- правильная установка освещения, которое не должно светить в глаза и оставлять блики на рабочем мониторе.

Для облегчения длительной работы эксперты рекомендуют использовать вспомогательное оборудование, например, особые очки, антибликовые мониторы, коврики для мыши и проч. Нарушения техники безопасности при работе на компьютере способны вызвать стойкие расстройства здоровья, которые потом будет трудно ликвидировать.

Режим труда и отдыха

Длительность непрерывного труда и отдыха для сотрудников, постоянно работающих за персональной офисной техникой, регламентирована на законодательном уровне. Соответствующие нормы приводятся в СанПиН 2.2.2/2.4.1340-03, который устанавливает обязательную продолжительность перерывов в такой работе. Они зависят от длины рабочей смены сотрудника и определяются так:

- при 8-часовой рабочей смене в зависимости от тяжести и напряженности труда работника общая продолжительность перерывов за рабочий день должна составлять от 50 до 90 минут;
- при 12-часовой рабочей смене – от 80 до 140 минут.

Проводить такие перерывы за монитором, тратя их на чтение новостей или онлайн-игры, нельзя. Работнику нужно посвятить их физическим упражнениям, короткой прогулке, гимнастике для глаз и другой активности.

Продолжительность указанных перерывов согласно действующему Трудовому кодексу включается в общую длительность рабочего времени: рабочий день сотрудника не продлевается за счет таких периодов отдыха. Также в этом время сотрудник не может привлекаться к другой работе. В дополнение к этим перерывам ему предоставляется стандартное время для обеда.

ЗАКЛЮЧЕНИЕ

В ходе ведения проекта было разработано приложение, которое автоматизирует учёт тематик, цель упрощения работы сотрудников учебного заведения считается успешно достигнутой.

Поэтапно были решены все поставленные задачи проекта:

- Был спроектирован подробный план приложения и его модулей, спланирована структура модулей приложения и логика взаимодействия между ними. Было составлено грамотное техническое задание
- Поэтапно были разработаны все необходимые модули приложения, в частности база данных и интерфейс.
- Все разработанные модули были протестированы и отлажены.
- Проект был подробно задокументирован, было написано руководство пользования программой.

Помимо основных задач проекта, были решены некоторые дополнительные задачи: были изучены инструменты разработки веб-интерфейсов, а так же основы веб-разработки с помощью модуля flask, отработана поставка приложения одиночным файлом. Данные навыки в последствии упростят работу с будущими проектами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ИНФОРМАЦИИ

1 Роббинс А. Изучаем vi и Vim. Не просто редакторы. 8-е изд./ Роббинс А., Ханна Э. СПБ.; Питер, 2023. - 528 с.

2 Документация python-модуля flask // Flask [Электронный ресурс]. - Режим доступа: https://flask.palletsprojects.com/en/3.0.x/ (дата обращения 08.05.2024)

3 Документация SQLite // SQLite.org [Электронный ресурс]. - Режим доступа: https://sqlite.org/ (дата обращения 05.05.2024)

4 Учебный портал для SQLite //SQLite Tutorial [Электронный ресурс]. - Режим доступа: https://www.sqlitetutorial.net/ (дата обращения 05.05.2024)

5 Руководство по работе с модулем pyinstaller // Pyinstaller [Электронный ресурс]. Режим доступа: https://pyinstaller.org/en/stable/ (дата обращения 29.05.2024)

6 Документация дизайнера шаблонов Jinja // Jinja [Электронный ресурс]. Режим доступа: https://jinja.palletsprojects.com/en/3.1.x/templates/ (дата обращения 09.05.2024)

7 Документация фреймворка bootstrap // Bootstrap [Электронный ресурс] Режим доступа: https://getbootstrap.com/docs/5.3/getting-started/introduction/ (дата обращения 14.05.2024)

8 Документация HTML // MDN web docs [Электронный ресурс] Режим доступа: https://developer.mozilla.org/en-US/docs/Web/HTML (дата обращения 09.05.2024)

9 Документация CSS // MDN web docs [Электронный ресурс] Режим доступа: https://developer.mozilla.org/en-US/docs/Web/CSS (дата обращения 13.05.2024)

10 Руководство по работе с формами // Digital Ocean [Электронный ресурс]
Режим доступа: https://www.digitalocean.com/community/tutorials/how-to-use-web-forms-in-a-flask-application (дата обращения 17.05.2024)

Приложение

Листинг 1 исходный код модуля app.py

```
import sqlite3
from sqlite3 import Error
import difflib
from flask import Flask, render_template, request, flash,
redirect

def compare_simmilarity(string1, string2):
    """ return simmiliarity score for string1 to string2
    :param string1:
    :param string2:
    :return: float simmiliarity score in range from 0 to 1
    """
    return difflib.SequenceMatcher(None,string1.lower(),
string2.lower()).ratio()

#READ ABOUT difflib.get_close_matches(search, heap, n=?,
cutoff=0.?)

def search_simmilar_themes(conn, search_string,
search_table_name="themes"):
    table_content = get_table(conn, search_table_name)[1]
    field_content = [(row[3], row[4]) for row in table_content]

    return difflib.get_close_matches(search_string,
field_content[1])

def create_connection(db_file):
    """ create a database connection to the SQLite database
        specified by db_file
    :param db_file: database file
    :return: Connection object or None
    """
    conn = None
    try:
        conn = sqlite3.connect(db_file)
        conn.row_factory = sqlite3.Row
        return conn
    except Error as e:
        print(e)

    return conn


def close_connection(conn):
    """ close connection do the SQLite database
    :param conn: Connection object
    :return: True or None
    """
    try:
        conn.close()
    except Error as e:
        print(e)
    else:
```

Продолжение листинга 1 исходный код модуля app.py

```
        return True


def get_themes(conn):
    """ get all themes from the "themes" table with student and
        discipline names joined in
    :param conn: Connection object
    :return: list of tuples (rows) with themes in format:
        ("discipline","student","theme")
    """
    cursor = conn.cursor()
    cursor.execute("""
                    SELECT
                        discipline.name as discipline,
                        student.name as student,
                        theme.type as type,
                        theme.content as content
                    FROM
                        theme
                    INNER JOIN discipline
                        ON theme.discipline_id = discipline.id
                    INNER JOIN student
                        ON theme.student_id = student.id;
                   """)
    column_names = list(map(lambda x: x[0], cursor.description))
    header = [description[0] for description in
cursor.description]
    return (header ,cursor.fetchall())


def get_disciplines(conn):
    """ get all disciplines from the "disciplines" table
    :param conn: Connection object
    :return: list of discipline names
    """
    cursor = conn.cursor()
    cursor.execute("""
                    SELECT
                        discipline.name as discipline
                    FROM
                        discipline
                   """)
    column_names = list(map(lambda x: x[0], cursor.description))
    header = [description[0] for description in
cursor.description]
    return (header ,cursor.fetchall())


def get_students(conn):
    """
    """
    cursor = conn.cursor()
    cursor.execute("""
                    SELECT
                        name
```

Продолжение листинга 1 исходный код модуля app.py

```
                        group_code
                    FROM
                        student
                    """)
    column_names = list(map(lambda x: x[0], cursor.description))
    header = [description[0] for description in
cursor.description]
    return (header ,cursor.fetchall())


def get_groups(conn):
    """
    """
    cursor = conn.cursor()
    cursor.execute("""
                    SELECT
                        "group".code,
                        qualification.name
                    FROM
                        "group"
                    INNER JOIN
                        qualification
                    ON "group".qualification_id =
qualification.id
                    """)
    column_names = list(map(lambda x: x[0], cursor.description))
    header = [description[0] for description in
cursor.description]
    return (header ,cursor.fetchall())


def get_qualifications(conn):
    """
    """
    cursor = conn.cursor()
    cursor.execute("""
                    SELECT
                        name
                    FROM
                        qualification
                    """)
    column_names = list(map(lambda x: x[0], cursor.description))
    header = [description[0] for description in
cursor.description]
    return (header ,cursor.fetchall())

def get_table(conn, table_name):
    """
    """
    if table_name == "themes":
        return get_themes(conn)
    elif table_name == "disciplines":
        return get_disciplines(conn)
    elif table_name == "students":
        return get_students(conn)
```

Продолжение листинга 1 исходный код модуля app.py

```
    elif table_name == "groups":
        return get_groups(conn)
    elif table_name == "qualifications":
        return get_qualifications(conn)
    else:
        return None


def add_qualification(conn, qualification):
    """add qualification into qualification table
    :param conn:
    :param qualification:
    :return: True or None
    """
    #TODO return ID instead of True
    #TODO try-except
    cursor = conn.cursor()
    cursor.execute("INSERT INTO qualification (name) VALUES
(?)", (qualification,))

    connection.commit()
    return True


def add_group(conn, group_code, qualification_id):
    """
    """
    #TODO try-except
    cursor = conn.cursor()
    cursor.execute("INSERT INTO group (code, qualification_id)
VALUES (?, ?)", (group_code, qualification_id))

    connection.commit()
    return True


def add_student(conn, name, group_code):
    #TODO return ID instead of True
    #TODO try-except
    cursor = conn.cursor()
    cursor.execute("INSERT INTO student (name, group_code)
VALUES (?, ?)", (name, group_code))

    connection.commit()
    return True

def add_discipline(conn, name):
    cursor = conn.cursor()
    cursor.execute("INSERT INTO discipline (name) VALUES (?)",
(name))

    connection.commit()
    return True

def add_theme(conn, discipline_id, student_id, type_, content):
```

Продолжение листинга 1 исходный код модуля app.py

```
    cursor = conn.cursor()
    cursor.execute("INSERT INTO discipline (discipline_id,
student_id, type, content) VALUES (?, ?, ?, ?, ?)", (conn,
discipline_id, student_id, type_, content))

    connection.commit()
    return True

def add_table(conn, table_name):
    """
    """
    if table_name == "themes":
        return add_theme(conn)
    elif table_name == "disciplines":
        return add_discipline(conn)
    elif table_name == "students":
        return add_student(conn)
    elif table_name == "groups":
        return add_groups(conn)
    elif table_name == "qualifications":
        return add_qualifications(conn)
    else:
        return None



database = "db-themes.db"
app = Flask(__name__)
app.config['SECRET_KEY'] =
'5708552aed9b6ad9e280c2651ac330f4795284b63655b485'

@app.route('/', methods=('GET', 'POST'))
def index():
    conn = create_connection(database)
    table_type = request.args.get('table')
    if not table_type:
        table_type = "themes"

    header, table_content = get_table(conn, table_type)
    if not header:
        flash(f"не существует таблицы %table_type")
    print(f"заголовки: {header}")
    print(f"содержимое: {table_content}")

    if request.method == "POST":
        search = request.form.items()
        print(f"список из формы поиска:")
        print(*search)
        if "search_request" in search:
            search_request = request.form['search_request']
            search_result = search_simmilar_themes(conn,
search_request)
            print(f"поисковый запрос: {search_request}")
```

Продолжение листинга 1 исходный код модуля app.py

```
    close_connection(conn)
    return render_template('index.html', header = header,
table_content =  table_content)

@app.route('/add/', methods=('GET', 'POST'))
def add():
    conn = create_connection(database)
    table_type = request.args.get('table')
    if not table_type:
        table_type = "themes"
    if request.method == 'POST':
        input_table = request.form.items()

        print(*input_table)
        for line in input_table:
            print(line)
            print(f"key = {line[0]}, value = {line[1]}")

    header, _ = get_table(conn, table_type)
    return render_template('add.html',  header = header, size =
10)
```

Листинг 2 исходный код базы данных

```
DROP TABLE IF EXISTS theme;
DROP TABLE IF EXISTS student;
DROP TABLE IF EXISTS discipline;
DROP TABLE IF EXISTS "group";
DROP TABLE IF EXISTS qualification;


CREATE TABLE IF NOT EXISTS qualification (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    name            TEXT
);

CREATE TABLE IF NOT EXISTS "group" (
    code            TEXT PRIMARY KEY,
    qualification_id   TEXT, --FK

    FOREIGN KEY (qualification_id)
        REFERENCES qualification (id)
);

CREATE TABLE IF NOT EXISTS discipline ( --Возможно тоже стоит
упростить до одного поля
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    name            TEXT
);

CREATE TABLE IF NOT EXISTS student (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    "name"          TEXT,
    group_code      TEXT, --FK
```

Продолжение листинга 2 исходный код базы данных

```
    FOREIGN KEY (group_code)
        REFERENCES "group" (code)

);


CREATE TABLE IF NOT EXISTS theme (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    discipline_id   INTEGER, --FK
    student_id      INTEGER, --FK
    type      TEXT,
    content   TEXT,

    FOREIGN KEY (discipline_id)
        REFERENCES discipline (id),
    FOREIGN KEY (student_id)
        REFERENCES student (id)
);
```

Листинг 3 html-шаблон base.html

```
<!doctype html>
<html lang="en">
  <head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1, shrink-to-fit=no">

    <!-- Bootstrap CSS -->
    <link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/boo
tstrap.min.css"
integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOh
cWr7x9JvoRxT2MZw1T" crossorigin="anonymous">
    <!-- My css -->
    <link rel="stylesheet" href=
"{{ url_for('static',filename='style.css') }}">

    <title>{% block title %} {% endblock %}</title>
  </head>
  <body>
    <nav class="navbar navbar-expand-md navbar-light bg-light">
        <a class="navbar-brand"
href="{{ url_for('index')}}">Индекс тем</a>
        <button class="navbar-toggler" type="button" data-
toggle="collapse" data-target="#navbarNav" aria-
controls="navbarNav" aria-expanded="false" aria-label="Toggle
navigation">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav">
            <li class="nav-item active">
```

Продолжение листинга 3 html-шаблон base.html

```
              <a class="nav-link"
href="{{url_for('index')}}">Просмотр</a>
            </li>
            <li class="nav-item active">
                <a class="nav-link"
href="{{ url_for('add') }}">Добавление</a>
            </li>
            </ul>
        </div>
    </nav>
    <div class="container">
        {% for message in get_flashed_messages() %}
            <div class="alert">{{ message }}</div>
        {% endfor %}
        {% block content %} {% endblock %}
    </div>

    <!-- Bootstrap JS -->
    <script
src="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/js/boots
trap.min.js" integrity="sha384-
JjSmVgyd0p3pXB1rRibZUAYoIIy6OrQ6VrjIEaFf/nJGzIxFDsf4x0xIM+B07jRM
" crossorigin="anonymous"></script>
  </body>
</html>
```

Листинг 4 html-шаблон index.html

```
{% extends 'base.html' %}

{% block content %}
    <h1>{% block title %} Просмотр тем {% endblock %}</h1>
    <form method="post">
        <div>
        <span><input type="text"
                    value="{{request.form['search_request']}}"
                    placeholder="Поиск...">
            </input></span`>
        <span><button type="submit",
name="search">Найти</button></span>
        <div>
    </form>
    <div>
        <ul>
        {% for result in search_results %}
            <li>{{result}}</li>
        {% endfor %}
        </ul>

    </div>
    <div style = "margin: auto; width: 50%;">
        <span><a href="?table=themes">Темы</a></span>
        <span><a href="?table=disciplines">Дисциплины</a></span>
        <span><a href="?table=students">Студенты</a></span>
        <span><a href="?table=groups">Группы</a></span>
```

Продолжение листинга 4 html-шаблон index.html

```
        <span><a
href="?table=qualifications">Квалификации</a></span>
    </div>
    <div style="overflow-x:auto; overflow-y:auto;">
        <table class="fixed_header">
            <thead>
                <tr>
                {% for atribute in header %}
                    <th>{{atribute}}</th>
                {% endfor %}
                </tr>
            </thead>
            <tbody>
                {% for row in table_content %}
                <tr>
                    {% for atribute in row %}
                    <td>{{atribute}}</td>
                    {% endfor %}
                </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
{% endblock %}
```

Листинг 5 html-шаблон add.html

```
{% extends 'base.html' %}

{% block content %}
    <h1>{% block title %}Добавление информации{% endblock
%}</h1>
    <form method="post">
        <h1>{{request.args.get('table')}}</h1>

        <div style = "margin: auto; width: 50%;">
            <span><a href="?table=themes">Темы</a></span>
            <span><a
href="?table=disciplines">Дисциплины</a></span>
            <span><a href="?table=students">Студенты</a></span>
            <span><a href="?table=groups">Группы</a></span>
            <span><a href="?
table=qualifications">Квалификации</a></span>
        </div>

        <div style="overflow-x:auto; overflow-y:auto;">
            <table class="fixed_header">
                <thead>
                    <tr>
                    {% for atribute in header %}
                        <th>{{atribute}}</th>
                    {% endfor %}
                    </tr>
                </thead>
                <tbody>
```

Продолжение листинга 5 html-шаблон add.html

```
                <!--{#{% for row in input_table %}#}-->
                <!-- maybe TODO namespace-->
                    {%for i in range(5) %}
                    <tr>
                        {% for atribute in header %}
                        {% set cell_name = i ~ atribute %}
                        <td><input type=text
                                   name="{{cell_name}}"

value="{{request.form[cell_name]}}"

placeholder="{{cell_name}}"></input></td>

                        {% endfor %}
                    </tr>
                    {% endfor %}
                </tbody>
            </table>
        </div>
        <button type="submit", name="submit">Добавить</button>
    </form>
{% endblock %}
```